AI 100 Final Project Report
Title: AI System Building with GenAI as a Cognitive Partner
Author: Sameer Ahmed
Course: AI 100

What I learned about GenAI in this project

For this final project, I reused my midterm diabetes classification model and then intentionally changed different parts of it to create bugs or unexpected behavior. The goal was not just to fix errors, but to understand what part of the system was affected and why. I used GenAI as a tool to check whether my thinking was actually specific and logical, instead of just guessing.

One of the biggest things I learned is that GenAI is more useful when I already have some idea of what is going on. If my reflection was vague, like saying the code was broken or the model was acting weird, the feedback was not very useful until I became more specific. But when I clearly explained what I changed, where the problem appeared, and what part of the pipeline I thought was involved, GenAI became much more helpful.

For example, one of my bug cases was changing the target column name from Outcome to Outcomes. That caused a KeyError before training even started. This made me realize that not every bug in an AI project is actually an AI bug. Sometimes the problem is just in the data loading step. In cases like this, GenAI was useful because it pushed me to identify the exact stage where the issue happened instead of blaming the whole program.

Another thing I learned is that GenAI works better as a reasoning partner than as a replacement for debugging. It helped most when I used it to check my explanation, not when I expected it to magically know what was wrong. I still had to run the code, read the traceback, compare outputs, and connect the bug to the actual change I made.

I also noticed that GenAI was more helpful for bugs that did not crash the code. For example, when I removed StandardScaler, removed stratify=y, changed the threshold from 0.5 to 0.9, or skipped model.eval(), the code still ran, but the results changed in important ways. In those cases, the issue was harder to notice because there was no obvious error message. GenAI helped me slow down and think about why the results changed instead of assuming it was just randomness.

At the same time, I learned that GenAI can only help if I give it good context. If I ignore the actual outputs, like the confusion matrix, accuracy, or loss values, then my reflection becomes weak. The best results came when I used the code output and the exact change together in my reasoning.

Overall, this project taught me that GenAI is useful for checking whether my thought process makes sense, but it does not replace understanding the system myself. It is helpful for improving reflection and debugging habits, but I still have to do the real work of reading errors, testing changes, and figuring out what part of the pipeline I affected.

What I learned about building an AI system in general

This project taught me that building an AI system is not just about the model. The whole pipeline matters, starting from the dataset and going all the way to evaluation.

The first thing I learned is that data setup matters a lot. If the label column is wrong, the whole system fails immediately. That sounds simple, but it showed me that even a good model cannot do anything if the data is not being read correctly.

I also learned that preprocessing is a very important part of the system. When I removed StandardScaler, the code still ran, but the model performed much worse. That showed me that preprocessing is not just an extra step. It can directly affect how well the optimizer learns from the data.

Another important lesson was that train and test splitting can change how results look. When I removed stratify=y, the class balance in the split changed, which made the metrics less reliable to compare against the original run. That helped me understand that performance numbers only mean something if the setup is fair and consistent.

This project also made tensor shapes feel much more real to me. Before this, shape mismatches were something I saw as just a PyTorch issue. But after breaking the label shape on purpose, I understood why the loss function needs outputs and targets in compatible forms. It made me pay more attention to dimensions instead of just hoping the code runs.

I learned the same thing with the model architecture. Changing the first layer from Linear(8,16) to Linear(10,16) broke the forward pass right away because the input no longer matched the feature count. That helped me see the direct link between the dataset and the model design.

Another big lesson was that the output layer and the loss function have to match. Using BCELoss with sigmoid output is one setup. Swapping to CrossEntropyLoss without changing the rest of the model caused bad behavior because the format was no longer compatible. This showed me that model design decisions are connected, and I cannot treat each part separately.

I also learned that just because code runs does not mean it is correct. Forgetting optimizer.zero_grad() did not crash the program, but it still changed the training behavior because gradients kept building up. In the same way, skipping model.eval() did not cause an error, but it changed test-time behavior because dropout stayed active. These kinds of bugs are easy to miss if I only look for crashes.

The threshold case also taught me something useful. When I changed the prediction threshold from 0.5 to 0.9, the model started predicting almost everything as negative. The weights were the same, but the final classification behavior changed a lot. That showed me that evaluation choices can strongly affect what kind of performance I think the model has.

The main lesson I am taking from this project is that an AI system is made of many connected parts: data, preprocessing, shapes, architecture, loss, optimizer, and evaluation. A small one-line change in any of these can cause a crash or quietly change the results. Because of that, I now see AI systems as something that has to be checked carefully from start to finish, not just at the model level.

This project helped me understand both debugging and system design in a more practical way. Instead of only trying to get a good accuracy score, I had to pay attention to how each part of the pipeline worked and what happened when I changed it. It also showed me that GenAI is most useful when I use it to improve my reasoning, not avoid it. After doing this project, I feel more confident reading errors, checking outputs, and understanding how different parts of an AI system connect to each other.